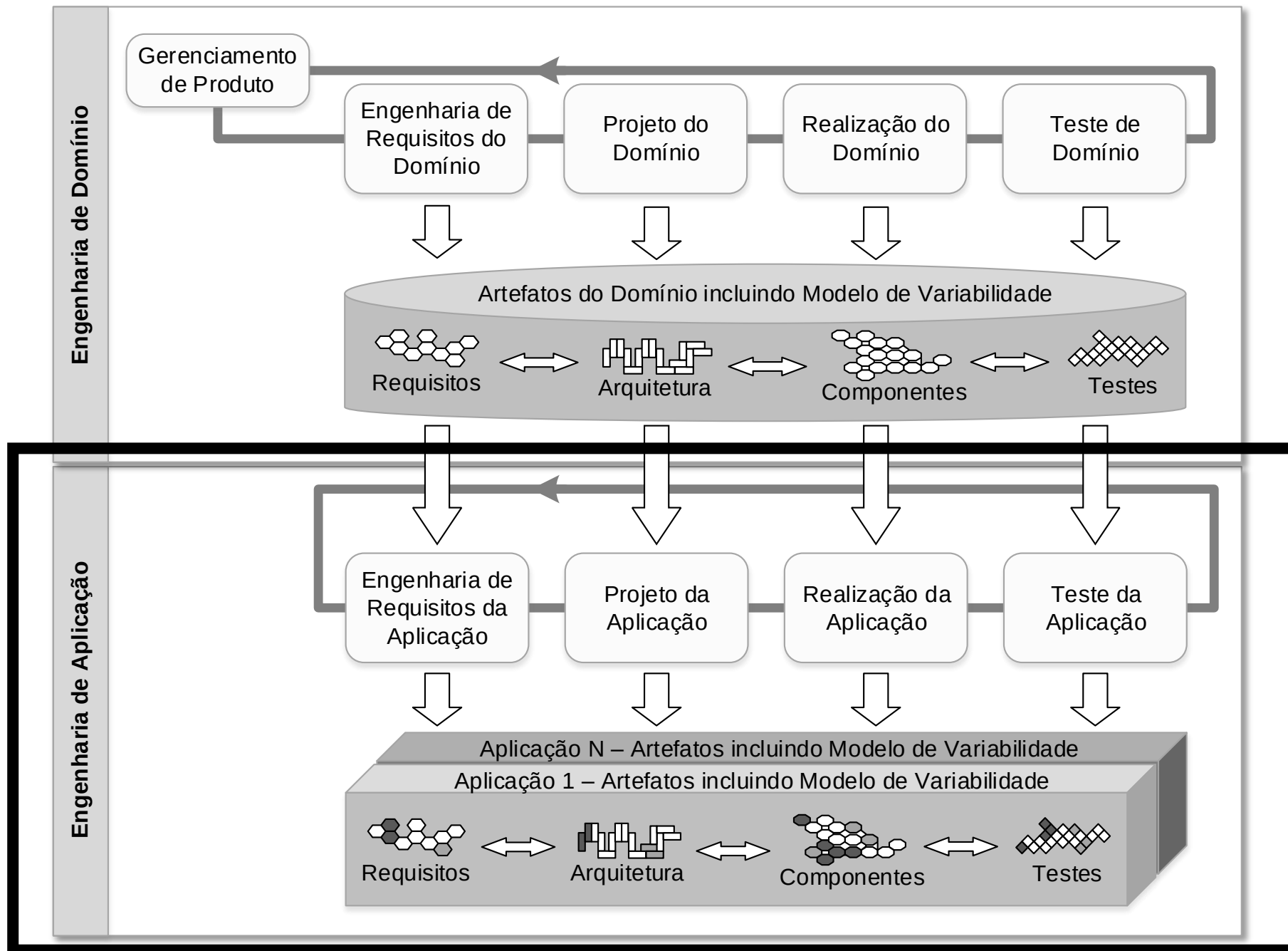




Design Patterns

(Padrões de Projetos)





Design patterns

		Propósito		
		1. Criação	2. Estrutura	3. Comportamento
Escopo	Classe	<i>Factory method</i>	<i>Class adapter</i>	<i>Interpreter</i> <i>Template method</i>
	Objeto	<i>Abstract factory</i> <i>Builder</i> <i>Prototype</i> <i>Singleton</i>	<i>Object adapter</i> <i>Bridge</i> <i>Composite</i> <i>Decorator</i> <i>Facade</i> <i>Flyweight</i> <i>Proxy</i>	<i>Chain of responsability</i> <i>Command</i> <i>Iterator</i> <i>Mediator</i> <i>Memento</i> <i>Observer</i> <i>State</i> <i>Strategy</i> <i>Visitor</i>

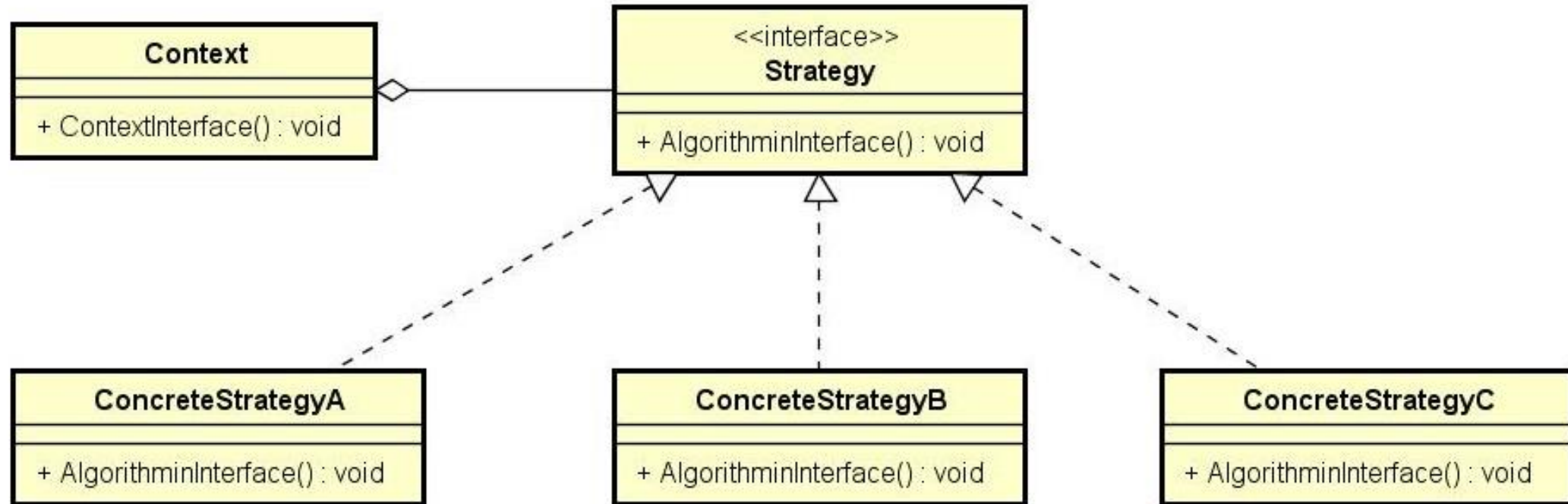
Baseado nos estudos de Erich Gamma; Categorização de design patterns por funcionalidade.



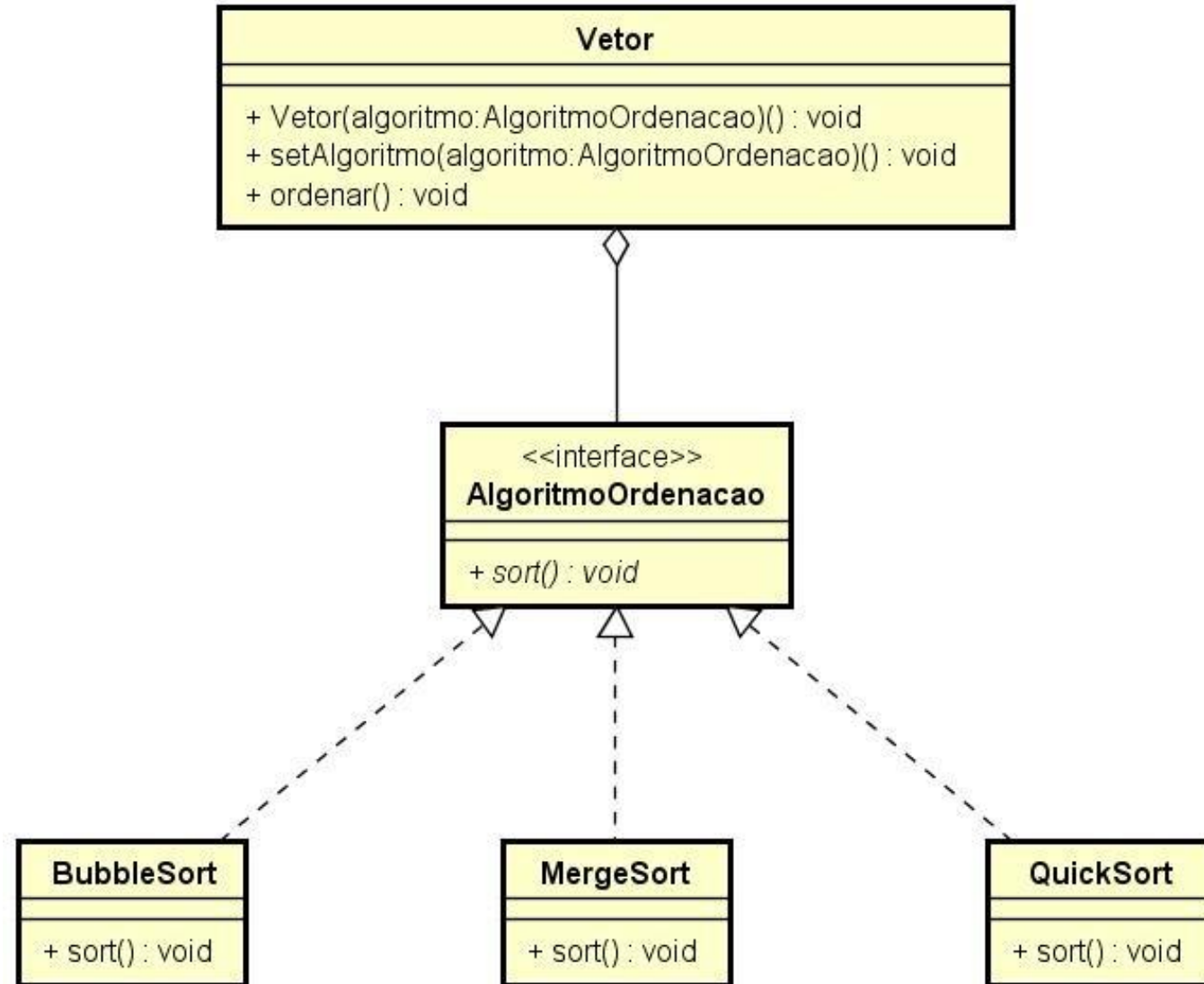
Strategy



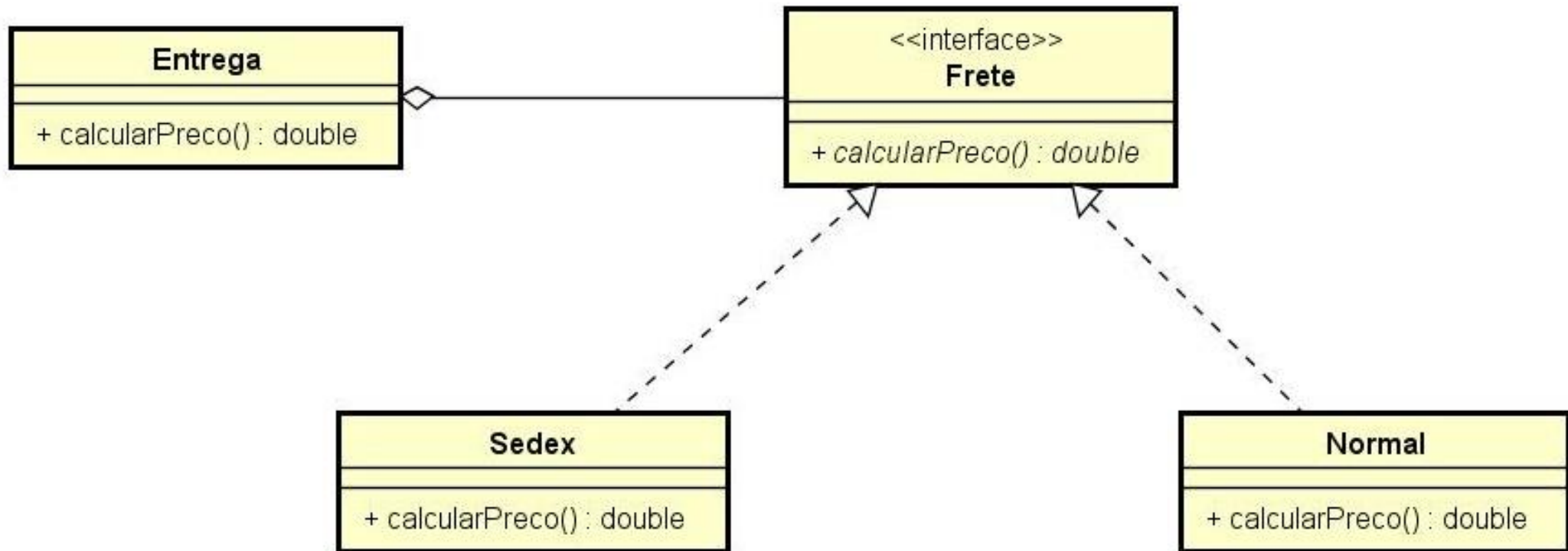
Strategy – Estrutura do padrão



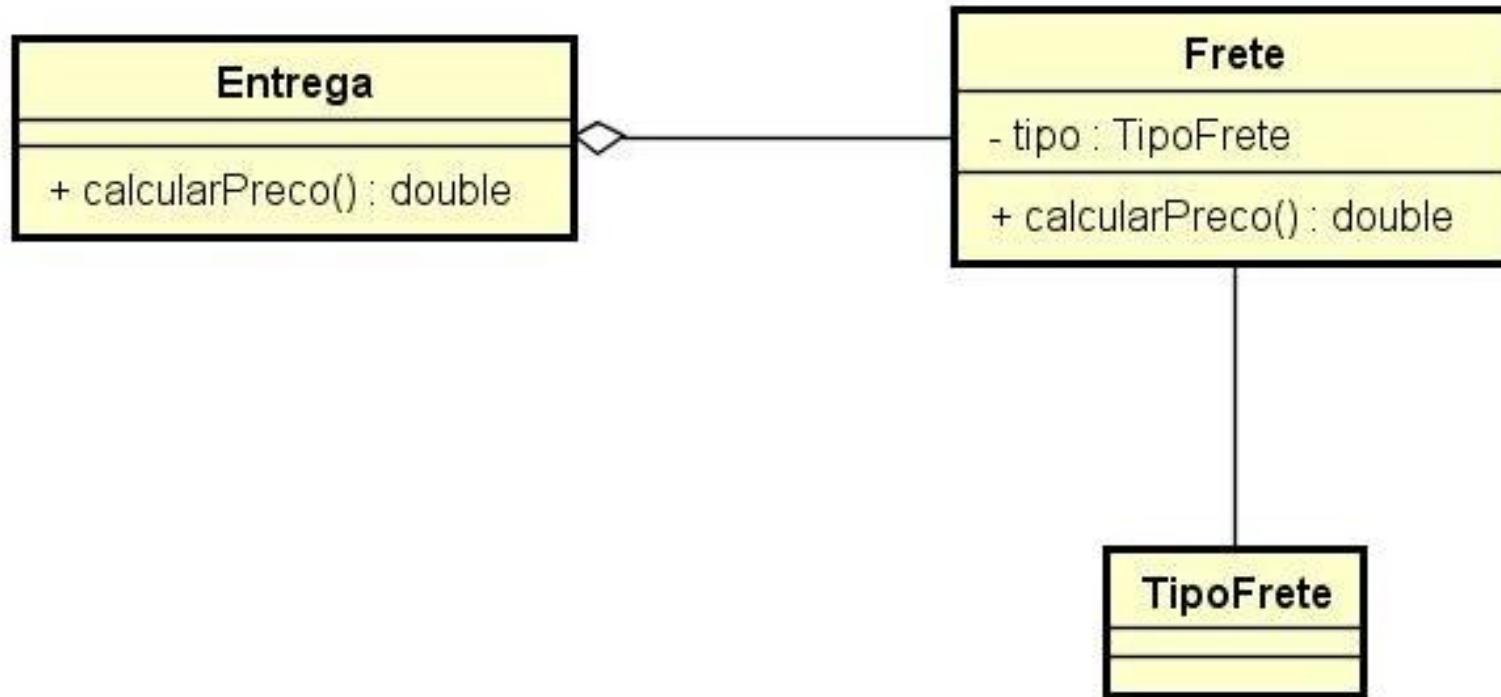
Strategy – Sistema controlador de algoritmos



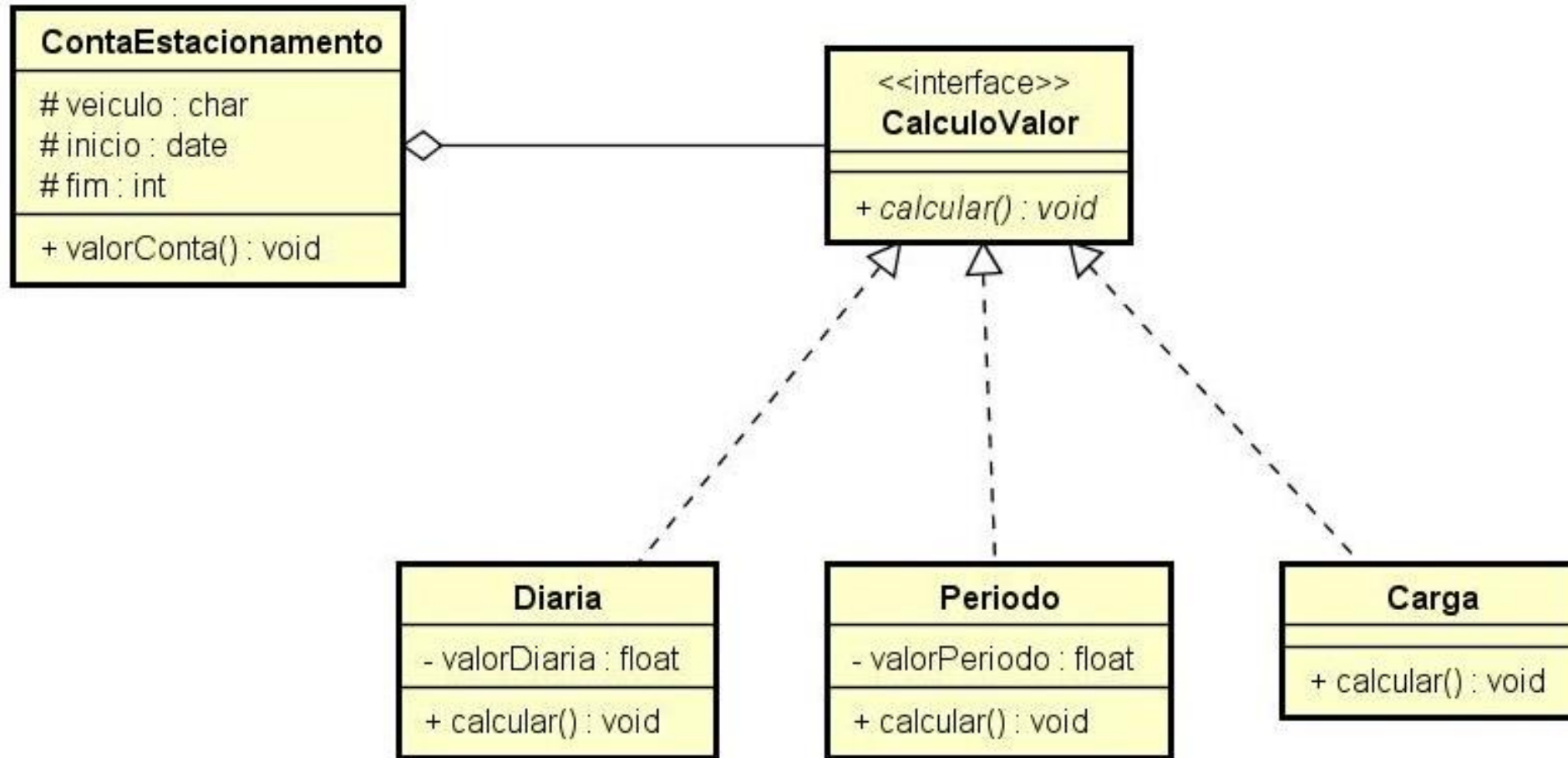
Strategy – Sistema de entrega de produtos



Strategy (não aplicado) – Sistema de entrega de produtos



Strategy – Sistema de estacionamento



Singleton



Singleton – Criação do objeto instancia

Singleton
- instancia : Singleton
+ construtor() : Singleton + getInstancia() : Singleton



Singleton – Instância única da classe

```
ModulodeGerenciamento.java  Aplicacao.java
1 package singleton;
2
3 public class ModulodeGerenciamento {
4
5     private static ModulodeGerenciamento instancia;
6
7     private ModulodeGerenciamento() {
8     }
9
10    public static ModulodeGerenciamento getInstance() {
11        if (instancia == null) {
12            instancia = new ModulodeGerenciamento();
13        }
14        return instancia;
15    }
16
17    public void Acesso() {
18    }
19 }
20 }
```



Singleton – Características do objeto

```
ModulodeGerenciamento.java  Aplicacao.java  ✕
1 package singleton;
2
3 public class Aplicacao {
4
5     public static void main(String[] args) {
6
7         ModulodeGerenciamento acesso01 = ModulodeGerenciamento.getInstance();
8         acesso01.Acesso();
9
10        ModulodeGerenciamento acesso02 = ModulodeGerenciamento.getInstance();
11        acesso02.Acesso();
12
13        ModulodeGerenciamento acesso03 = ModulodeGerenciamento.getInstance();
14        acesso03.Acesso();
15
16    }
17 }
18
```



Singleton – DAO (Data Access Object)

```
25 public class FuncionarioDAO {
26
27     private static Connection c;
28
29     public FuncionarioDAO() {
30         FuncionarioDAO.c = ConnectionFactory.getConnection();
31     }
32
33     public static void insereFuncionario(Funcionario fun) {
34         String sql = "INSERT INTO funcionario(nome, cargo, data, tipo, salario, telefone)"
35             + "VALUES (?, ?, ?, ?, ?, ?)";
36
37         try {
38             PreparedStatement stmt = c.prepareStatement(sql);
39             stmt.setString(1, fun.getNome());
40             stmt.setString(2, fun.getCargo());
41             stmt.setString(3, fun.getData());
42             stmt.setString(4, fun.getTipo());
43             stmt.setDouble(5, fun.getSalario());
44             stmt.setInt(6, fun.getTelefone());
45             stmt.execute();
46             stmt.close();
```



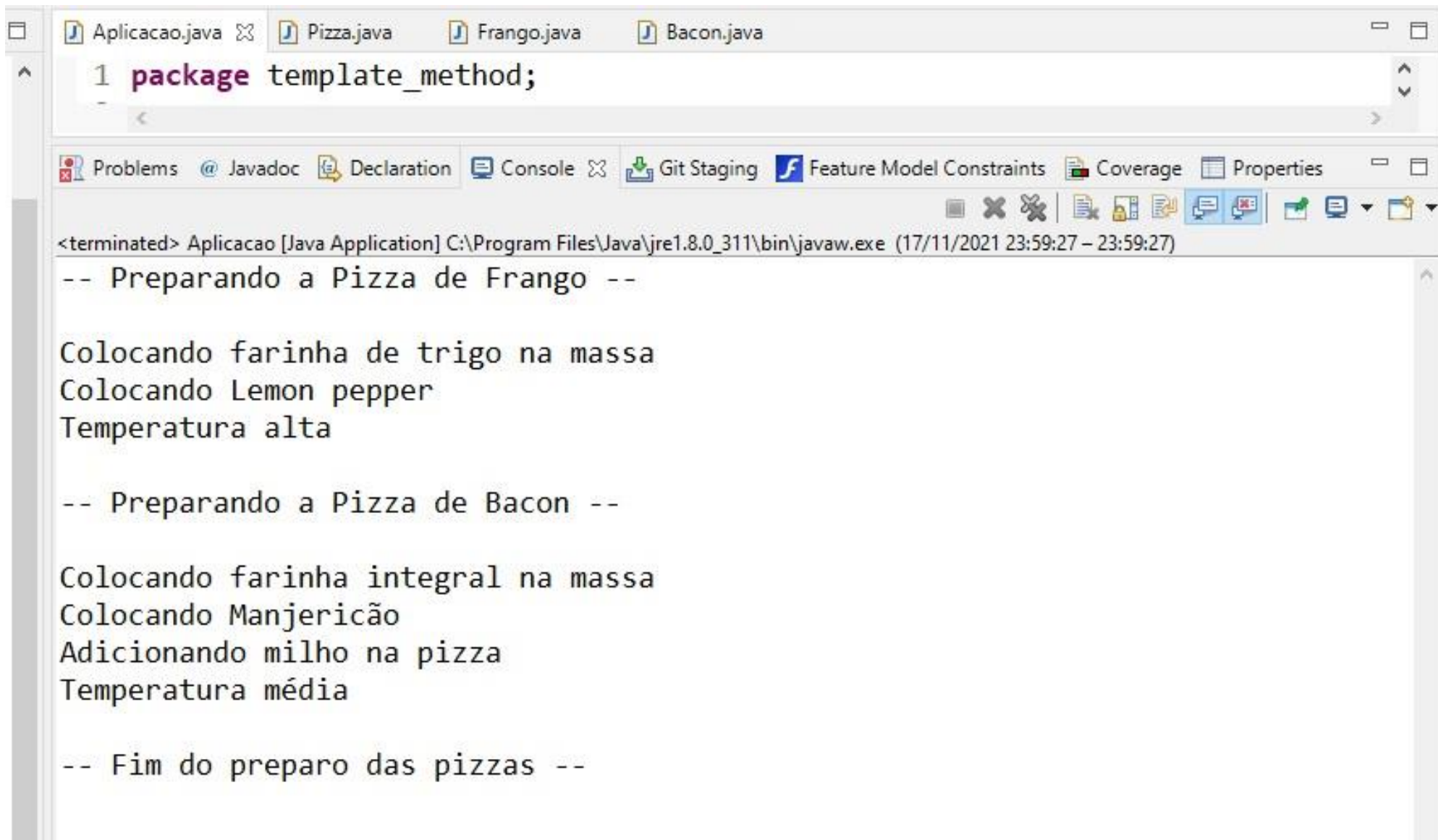
Singleton – Connection Factory

```
7
8  import java.sql.Connection;
9  import java.sql.DriverManager;
10 import java.sql.SQLException;
11
12 /**
13  *
14  * @author Tiago
15  *
16  * localhost e o endereço de loopback
17  */
18 public class ConnectionFactory {
19
20     public static Connection getConnection() {
21         Connection c = null;
22         try {
23             c = DriverManager.getConnection("jdbc:postgresql://127.0.0.1/Tabelas", "postgres", "12345");
24         } catch (SQLException e) {
25             System.out.println(e.getMessage());
26         }
27         return c;
28     }
29 }
```

Template Method



Template Method – Exibição



The screenshot shows an IDE window with four tabs: `Aplicacao.java`, `Pizza.java`, `Frango.java`, and `Bacon.java`. The `Aplicacao.java` tab is active, displaying the following code:

```
1 package template_method;
```

Below the code editor, the console output is visible, showing the execution of a Java application. The output is as follows:

```
<terminated> Aplicacao [Java Application] C:\Program Files\Java\jre1.8.0_311\bin\javaw.exe (17/11/2021 23:59:27 - 23:59:27)

-- Preparando a Pizza de Frango --

Colocando farinha de trigo na massa
Colocando Lemon pepper
Temperatura alta

-- Preparando a Pizza de Bacon --

Colocando farinha integral na massa
Colocando Manjericão
Adicionando milho na pizza
Temperatura média

-- Fim do preparo das pizzas --
```



Template Method – Aplicação

```
Aplicacao.java Pizza.java Frango.java Bacon.java
1 package template_method;
2
3 public class Aplicacao {
4
5     public static void main(String[] args) {
6
7         System.out.println("-- Preparando a Pizza de Frango --");
8         System.out.println("");
9         Frango f = new Frango();
10        f.fazerPizza();
11        System.out.println("");
12
13        System.out.println("-- Preparando a Pizza de Bacon --");
14        System.out.println("");
15        Bacon b = new Bacon();
16        b.fazerPizza();
17
18        System.out.println("");
19        System.out.println("-- Fim do preparo das pizzas --");
20    }
21
22 }
```



Template Method – Classe Pizza

```
Aplicacao.java Pizza.java Frango.java Bacon.java
1 package template_method;
2
3 public abstract class Pizza {
4
5     public void fazerPizza() {
6
7         colocarFarinha();
8         colocarTempero();
9
10        if (acrescentarMilho()){
11            colocarMilho();
12        }
13
14        colocaraoForno();
15    }
16
17    public abstract void colocarFarinha();
18    public abstract void colocarTempero();
19    public abstract boolean acrescentarMilho();
20    public abstract void colocarMilho();
21    public abstract void colocaraoForno();
22 }
23
```



Template Method

Classe Frango

```
Aplicacao.java Pizza.java Frango.java Bacon.java
2
3 public class Frango extends Pizza {
4
5     @Override
6     public void colocarFarinha() {
7         System.out.println("Colocando farinha de trigo na massa");
8     }
9
10    @Override
11    public void colocarTempero() {
12        System.out.println("Colocando Lemon pepper");
13    }
14
15    @Override
16    public boolean acrescentarMilho() {
17        return false;
18    }
19
20    @Override
21    public void colocarMilho() {
22        System.out.println("Não aplicado");
23    }
24
25    @Override
26    public void colocaraoForno() {
27        System.out.println("Temperatura alta");
28    }
29 }
30
```



Template Method

Classe Bacon

```
Aplicacao.java Pizza.java Frango.java Bacon.java
2
3 public class Bacon extends Pizza {
4
5     @Override
6     public void colocarFarinha() {
7         System.out.println("Colocando farinha integral na massa");
8     }
9
10    @Override
11    public void colocarTempero() {
12        System.out.println("Colocando Manjeriçã");
13    }
14
15    @Override
16    public boolean acrescentarMilho() {
17        return true;
18    }
19
20    @Override
21    public void colocarMilho() {
22        System.out.println("Adicionando milho na pizza");
23    }
24
25    @Override
26    public void colocaraoForno() {
27        System.out.println("Temperatura média");
28    }
29
30 }
```



Projeto da equipe

Você e sua equipe devem aplicar 3 padrões de projetos focados no **reuso de software**. Não é necessário a modelagem do padrão, vocês devem desenvolver e entregar apenas a **codificação**. Vocês podem escolher a opção 01 ou 02.

Mas antes vocês devem codificar as principais classes (ao menos 10) com métodos e atributos.

- **Opção 01**
 - Singleton – Codificar 2 exemplos
 - Template Method – Codificar 3 exemplos
 - + um novo design patterns – Codificar 3 exemplos
- **Opção 02**
 - 3 outros padrões de projetos – Codificar 2 exemplos de cada





Tiago Navarro

tiago.navarro@pucpr.br

